

LABORATORIO 06

¿Y quién repartió el trabajo?

Producción, consumo y grupos.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 50 minutos · necesitas cinco terminales

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber visto, con tus ojos y en terminales distintas, **dos procesos que leen lo mismo y se reparten el trabajo**, y **dos procesos que leen lo mismo y lo duplican**. La única diferencia entre los dos casos va a ser **una palabra del comando**.

Este es el concepto que más cuesta de Kafka. No se parece a nada de lo que viste hasta ahora, y por fuera los dos casos se ven idénticos.

LO QUE ESTE LABORATORIO DEMUESTRA

Dos consumidores del mismo grupo se reparten el trabajo. Dos grupos distintos leen todo cada uno.

Y hay una consecuencia que cuesta plata, y que también vas a ver en pantalla.

LA CONSECUENCIA

Más consumidores que particiones no acelera nada. El sobrante mira.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Cinco terminales de Git Bash	Cuatro para los consumidores y una para trabajar. Las vas a necesitar todas
Las cinco en la carpeta del laboratorio	En cada una, <code>cd Capitulo_3/lab-06-produccion-consumo-cli</code>
El repositorio del curso actualizado	<code>git pull</code> antes de empezar
Los puertos 9092, 9093, 9094 y 8090 libres	El paso 1 te avisa si no lo están

CÓMO VAMOS A LLAMAR A LAS TERMINALES

Para que no te pierdas, cada terminal tiene nombre en toda la guía. **Ponles ese nombre en la pestaña si tu terminal lo permite.**

Terminal	Para qué
A, B, C, D	Los consumidores. Se quedan corriendo y no las vas a poder usar para otra cosa
T, la de trabajo	Desde aquí se crea el tópico, se escriben mensajes y se consulta el estado de los grupos

El caso

El proceso que valida comprobantes recibidos en **NovaTech Logistics** empieza a quedarse corto. Llegan más rápido de lo que se validan, la cola crece, y a las seis de la tarde hay cuatro horas de atraso.

La solución que propone cualquiera es obvia. **Levantemos un segundo proceso.**

Y ahí aparece la pregunta que nadie sabe contestar en la reunión. **Si levanto un segundo validador, ¿se reparten los comprobantes, o los dos validan los mismos?**

Las dos respuestas son plausibles y las consecuencias son opuestas. Si se reparten, el atraso se corta a la mitad y el problema está resuelto. Si los dos leen todo, cada comprobante se valida dos veces. En un proceso tributario eso no es un poco más lento. Es un comprobante contabilizado dos veces, y el atraso además sigue igual.

Lo peor es que **por fuera se ven idénticos**. Dos procesos, la misma fuente de datos, el mismo comando. Nada en la pantalla te dice cuál de los dos mundos estás habitando, hasta que alguien revisa las cifras a fin de mes.

LA METÁFORA · HOY CRUZAMOS A LA COCINA

Kafka sigue siendo **el restaurante**. El **mozo** es el **broker**, el **tipo de comanda** es el **tópico**, los **sectores del salón** son las **particiones**, el **espiche** donde se clavan las comandas es el **segmento**.

Hasta ahora miramos siempre el lado del mozo. Quién anota, dónde se guarda, cuánto dura. **Hoy cruzamos a la cocina**, y entran dos piezas nuevas.

El **cocinero** que va sacando comandas del espiche para prepararlas es el **consumidor**. Y la **brigada**, o sea los cocineros que trabajan la misma carta y se reparten las comandas entre ellos, es el **grupo de consumo**.

Con eso, el problema del caso se dice en una frase de restaurante. Si entra un segundo cocinero, **¿se reparte las comandas con el primero, o prepara todos los platos otra vez?**

LA RESPUESTA DEL RESTAURANTE ES LA DE KAFKA

Depende de si los pusiste en la misma brigada. **Dos cocineros de la misma brigada se reparten el espiche.**

Dos brigadas distintas, como la cocina y el equipo de auditoría que revisa cada comanda, **preparan cada una todas las comandas**, porque están haciendo cosas distintas con ellas. 🍴
La palabra que lo decide es `--group`.

Levantar el clúster y preparar las terminales

Qué vamos a hacer

Este laboratorio usa el mismo clúster de tres brokers del laboratorio 05. Si lo dejaste arriba, no hace falta que lo bajes. Si vienes de cero, hay que levantarlo.

Los comandos, en la terminal T

```
cd Capitulo_3/lab-06-produccion-consumo-cli
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

`cd Capitulo_3/lab-06-produccion-consumo-cli` — la carpeta de este laboratorio. **Es distinta de la del 05**, aunque el clúster que levanta sea equivalente

`chmod +x bin/*.sh kafka-cli/*.sh` — permiso de ejecución para los scripts. En Git Bash a veces se pierde al clonar

`bin/start-lab.sh` — levanta los tres brokers y la interfaz web. **Antes limpia cinco nombres de contenedor** que puedan haber quedado de otro lab del curso, que son `kafka-broker-1`, `-2`, `-3`, `kafbat-ui` y `gps-producer`. **No limpia `kafka-broker`, el del lab 01, ni `kafka-broker-4`, el del lab 08**, así que si vienes de uno de esos dos bájalo tú con `docker compose -p novatech-labNN down -`

v

LO QUE VAS A VER · RECORTADO, EL ORIGINAL TRAE UN BANNER GRANDE

```
[1/4] Levantando contenedores del clúster NovaTech...
```

```
✓ Container kafka-broker-1           Healthy
✓ Container kafka-broker-2           Healthy
✓ Container kafka-broker-3           Healthy
✓ Container kafbat-ui                 Started
```

```
[2/4] Esperando a que los brokers estén operativos...
```

```
✓ 3/3 brokers operativos
```

```
CLÚSTER NOVATECH OPERATIVO
```

Ahora abre las otras cuatro terminales

Abre cuatro ventanas más de Git Bash, y en cada una entra a la carpeta del laboratorio:

```
cd Capitulo_3/lab-06-produccion-consumo-cli
```

VAS BIEN SI...

Tienes **cinco terminales abiertas**, todas en la carpeta del laboratorio, y en la terminal T salió `3/3 brokers operativos` .

Compruébalo con `docker ps --filter name=kafka-broker` . Tres líneas, las tres con `Up` y `(healthy)` .

SI ALGO FALLA EN ESTE PASO

Dice `[ERROR] Docker no está corriendo` . Abre Docker Desktop, espera a que la ballena se calme y repite.

Dice `port is already allocated` . Algo usa el 9092, el 9093, el 9094 o el 8090. Mira qué con `docker ps` y bájalo.

Dice `Timeout: solo 2/3 brokers están operativos` . Casi siempre es memoria. Sube Docker a 6 GB y repite `bin/start-lab.sh` .

El tópico de tres sectores

Qué vamos a hacer

CONCEPTO · PARTICIÓN

Cada uno de los **pedazos en que se corta un tópico**. Son los **sectores del salón**, y existen exactamente para esto, para que el trabajo se pueda repartir entre varios.

Necesitamos un tópico con **3 particiones**. Tres es el número más chico que permite ver un reparto desigual entre dos miembros, o sea 2 y 1. Y también es el más chico que deja ver, con una cuarta terminal, a un miembro sin nada que hacer.

HOY LOS COMANDOS VAN CRUDOS, SIN ENVOLTORIO

En el laboratorio 05 usamos los envoltorios de `kafka-cli/`. Hoy no.

El motivo es concreto. El envoltorio `consume-as-group.sh` trae el tópico fijo por dentro, uno de 6 particiones, y no acepta `--topic`. Hoy necesitamos uno de 3 particiones para que el techo se vea con cuatro terminales en vez de siete.

No es una pérdida. **Es exactamente lo que vas a escribir en el servidor de SUNAT**, donde no hay envoltorios ni Docker.

El comando, en la terminal T

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 \
  --create --topic novatech.validacion \
  --partitions 3 --replication-factor 3
```

`docker exec` — ejecuta el comando **dentro** de un contenedor que ya corre. **En el servidor de SUNAT esta parte no existe**

`kafka-broker-1` — en cuál de los tres contenedores. Cualquiera sirve

`kafka-topics` — la herramienta que administra tópicos

`--bootstrap-server kafka-broker-1:29092` — el puerto **interno**, porque le hablamos desde dentro de la red de Docker. Desde tu máquina sería `localhost:9092`

`--create --topic novatech.validacion` — crear, y con qué nombre

--partitions 3 — el número del laboratorio. Es el techo de paralelismo que vamos a chocar en el paso 6

--replication-factor 3 — tres copias de cada partición, como el resto del clúster

LO QUE VAS A VER

```
WARNING: Due to limitations in metric names, topics with a period ('.') or
underscore ('_') could collide. To avoid issues it is best to use either,
but not both.
Created topic novatech.validacion.
```

ESE WARNING NO ES UN ERROR

Sale siempre que el nombre del tópico lleva puntos, y todos los del curso los llevan. Kafka avisa que un nombre que mezcla puntos y guiones bajos podría chocar en sus métricas internas. **El tópico se creó igual**, y la línea `Created topic` lo confirma.

Verifícalo

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --topic novatech.validacion
```

--describe — solo consulta. No modifica nada

LA PRIMERA LÍNEA DE LA SALIDA

```
Topic: novatech.validacion TopicId: S4mA2u1ISYKQCwSHGBo4uQ
PartitionCount: 3 ReplicationFactor: 3 Configs: min.insync.replicas=2
```

VAS BIEN SI...

`PartitionCount` dice **3**. Ese número es el protagonista de todo lo que sigue. Anótalo.

Debajo salen tres líneas más, una por partición, numeradas 0, 1 y 2.

SI DICE QUE EL TÓPICO YA EXISTE

Te pasa si repites el laboratorio. Bórralo y créalo de nuevo, porque necesitas los contadores en cero:

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --delete --topic novatech.validacion
```

Un cocinero solo

Qué vamos a hacer

CONCEPTO · CONSUMIDOR

Un proceso que **lee** de un tópico. Puede estar en un grupo o no. Es el cocinero.

CONCEPTO · GRUPO DE CONSUMO

El conjunto de consumidores que Kafka trata como **un solo trabajo repartido entre varios**. Se declara con `--group <nombre>`. Kafka le asigna a cada miembro un puñado de particiones, y **cada mensaje lo lee exactamente un miembro del grupo**. Es la brigada.

Lanzamos **un** consumidor dentro del grupo `validacion` y miramos qué le tocó. Con un solo miembro la respuesta no tiene misterio, pero fija la línea base contra la que vas a comparar todo lo demás.

UNA DIFERENCIA IMPORTANTE CON EL LABORATORIO 05

Los consumidores de hoy no terminan solos. Se quedan corriendo, esperando mensajes, hasta que tú los cortes con `Ctrl+C`.

Eso no es un error ni rompe nada. **Es lo que hace un consumidor de verdad.** Cortarlo con `Ctrl+C` es la forma correcta de sacarlo, y en el paso 7 lo vamos a usar a propósito.

El comando, en la terminal A

```
docker exec -it kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion \
  --group validacion \
  --consumer-property client.id=cons-A \
  --property print.partition=true \
  --property print.key=true --property key.separator='|'
```

`docker exec -it` — la `-i` mantiene la entrada abierta y la `-t` le da una terminal. **Juntas son las que hacen que `Ctrl+C` funcione** y que las líneas aparezcan a medida que llegan

`kafka-console-consumer` — el consumidor de línea de comandos que trae Kafka

`--topic novatech.validacion` — de qué tópico se lee

`--group validacion` — la palabra del laboratorio. Te unes a ese grupo, y Kafka te asigna particiones en vez de darte todo

`--consumer-property client.id=cons-A` — le pone nombre a este consumidor. Sin esto Kafka le inventa uno como `console-consumer-40f2d1ba...` y los listados se vuelven ilegibles

`--property print.partition=true` — imprime de qué partición vino cada mensaje. Sin esto no se ve el reparto, que es todo el laboratorio

`--property print.key=true` — imprime también la clave del mensaje

`--property key.separator='|'` — con qué carácter se separan clave y valor en pantalla

LO QUE VAS A VER

Nada. El cursor se queda esperando.

ESO ES CORRECTO · DÉJALO CORRIENDO

Todavía no hay mensajes en el tópico. El consumidor está conectado y esperando, que es lo que hace un consumidor.

Antes de quedarse quieto imprime dos avisos. **No son errores y el comando funciona igual.**

Option `--consumer-property` is deprecated and will be removed in a future version. Use `--command-property` instead.

Option `--property` is deprecated and will be removed in a future version. Use `--formatter-property` instead.

Ahora pregunta cómo quedó el grupo, desde la terminal T

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group validacion
```

`kafka-consumer-groups` — la herramienta que responde por los grupos. **Quién está, qué le tocó, y cuánto le falta por leer**

`--describe --group validacion` — el detalle de ese grupo. Solo consulta, no modifica nada

LO QUE VAS A VER · RECORTADO A LO ANCHO

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CLIENT-ID
validacion	novatech.validacion	0	0	0	0	cons-A
validacion	novatech.validacion	1	0	0	0	cons-A
validacion	novatech.validacion	2	0	0	0	cons-A

Una línea por partición, y en las tres el mismo `CLIENT-ID` . `cons-A` **se quedó con las tres**. Es el único cocinero, así que atiende el salón entero.

CONCEPTO · LAG

Cuántos mensajes escritos **todavía no leyó** un grupo. Es la resta `LOG-END-OFFSET` menos `CURRENT-OFFSET` , la misma cuenta de los extremos del laboratorio 05. **Un `LAG` que crece es la señal de que el consumo va más lento que la producción**, o sea el atraso de las cuatro horas del caso, medido.

Columna	Qué dice
<code>PARTITION</code>	Cada partición lleva su cuenta por separado
<code>CURRENT-OFFSET</code>	Hasta dónde leyó el grupo en esa partición
<code>LOG-END-OFFSET</code>	Hasta dónde escribió el productor
<code>LAG</code>	Lo que falta por leer, la resta de los dos anteriores. Es el número que se vigila en producción
<code>CONSUMER-ID</code>	Qué miembro atiende esa partición. Es un identificador largo que Kafka genera
<code>CLIENT-ID</code>	El nombre que le pusimos con <code>client.id</code> . Por eso se lee <code>cons-A</code>

VAS BIEN SI...

Salen **tres líneas**, una por partición, y en las tres el `CLIENT-ID` dice `cons-A` .

Los `LAG` están en 0 porque todavía no hay nada escrito.

SI DICE `CONSUMER GROUP 'VALIDACION' DOES NOT EXIST`

El consumidor de la terminal A no llegó a unirse al grupo. Mira esa terminal.

Si ahí hay un error, casi siempre es el nombre del tópico mal escrito. Compruébalo con `docker exec kafka-broker-1 kafka-topics --bootstrap-server kafka-broker-1:29092 --list` .

SI EL `CLIENT-ID` SALE COMO `CONSOLE-CONSUMER-40F2D1BA...`

Se te perdió el `--consumer-property client.id=cons-A` . No rompe el laboratorio, pero vas a tener que ir comparando identificadores largos. **Corta con `Ctrl+C` y relánzalo completo**, se agradece después.

Entra el segundo cocinero

Qué vamos a hacer

Este es el paso que contesta la pregunta de la reunión. Sumamos un segundo consumidor **al mismo grupo**, escribimos seis comprobantes, y contamos cuántos recibió cada uno.

El comando, en la terminal B

Es el mismo de antes, cambiando solo el `client.id`:

```
docker exec -it kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion \
  --group validacion \
  --consumer-property client.id=cons-B \
  --property print.partition=true \
  --property print.key=true --property key.separator='|'
```

--group validacion — idéntico al de la terminal A. Ahí está todo el experimento

client.id=cons-B — lo único que cambia, para poder distinguirlos en los listados

ESPERA UNOS 20 SEGUNDOS ANTES DE SEGUIR

Kafka tiene que **rehacer el reparto** ahora que hay dos miembros. Si preguntas de inmediato vas a ver el reparto viejo, o un grupo en estado `PreparingRebalance`. Dale tiempo.

Vuelve a preguntar, desde la terminal T

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group validacion
```

LO QUE VAS A VER · RECORTADO A LO ANCHO

GROUP	TOPIC	PARTITION	...	CLIENT-ID
validacion	novatech.validacion	0	...	cons-A
validacion	novatech.validacion	1	...	cons-A
validacion	novatech.validacion	2	...	cons-B

Las tres particiones siguen repartidas, pero ahora entre dos nombres. **cons-A** se quedó con la 0 y la 1, y **cons-B** con la 2. Nadie las comparte.

A TI TE PUEDE TOCAR AL REVÉS

Que **cons-B** se lleve dos y **cons-A** una da exactamente igual. **Lo que no puede pasar es que una partición aparezca dos veces.**

Ahora los seis comprobantes, desde la terminal T

```
for i in 1 2 3 4 5 6; do
  echo "RUC-2010006660${i}:comprobante_${i}" | \
  docker exec -i kafka-broker-1 kafka-console-producer \
    --bootstrap-server kafka-broker-1:29092 \
    --topic novatech.validacion \
    --property parse.key=true --property key.separator=:
done
```

for i in 1 2 3 4 5 6; do ... done — un bucle. Seis comprobantes, uno por vuelta. **Cada vuelta abre un productor nuevo**, por eso tarda unos segundos

echo "RUC-2010006660\${i}:comprobante_\${i}" — la línea que se le manda al productor. Clave y valor separados por **:**

| — la tubería. Toma lo que imprimió **echo** y se lo mete al productor por la entrada estándar

docker exec -i — aquí va **sin la -t**. No queremos una terminal, queremos meterle texto por la entrada

kafka-console-producer — el productor de línea de comandos que trae Kafka

--property parse.key=true — **que parta cada línea en clave y valor**. Sin esto la clave viaja dentro del valor, y todos los mensajes caen en la misma partición

--property key.separator=: — por qué carácter la parte. Lo anterior al primer **:** es la clave

LO QUE VAS A VER EN LA TERMINAL T

Nada por cada mensaje. Su silencio es el éxito.

Lo interesante pasa en las terminales A y B. Esto salió en una corrida real:

TERMINAL A · CONS-A

```
Partition:1|RUC-20100066601|comprobante_1
Partition:0|RUC-20100066602|comprobante_2
Partition:1|RUC-20100066603|comprobante_3
Partition:0|RUC-20100066605|comprobante_5
```

TERMINAL B · CONS-B

```
Partition:2|RUC-20100066604|comprobante_4
Partition:2|RUC-20100066606|comprobante_6
```

Cuatro y dos. Suman seis. Y ni uno solo aparece en las dos terminales.

Esa es la primera mitad de la afirmación, y ya contesta la reunión del caso. **Dos validadores del mismo grupo se reparten el trabajo, no lo duplican.**

OJO CON EL «CUATRO Y DOS»

El reparto es de particiones, no de mensajes. `cons-A` tenía dos particiones y `cons-B` una, así que era esperable que a A le tocara más. Y dentro de eso, en qué partición cae cada comprobante lo decide el *hash* de su clave, que con seis mensajes no reparte parejo.

Con seis mensajes, un reparto de seis y cero es perfectamente posible, y no sería un fallo. Significaría que las seis claves cayeron en particiones del mismo miembro. Lo que demuestra el laboratorio no es que el reparto sea parejo. Es que **cada mensaje lo recibe uno solo.**

VAS BIEN SI...

Entre las dos terminales sumas exactamente seis mensajes, y ningún `comprobante_N` aparece dos veces.

Ese es el resultado, sea cual sea el reparto que te haya tocado.

SI EL MISMO COMPROBANTE SALE EN LAS DOS TERMINALES

Revisa el `--group` de las dos. **Tienen que decir exactamente `validacion`, con la misma ortografía.** Si una dice `validacion` y la otra `validación` con tilde, son dos grupos distintos, y por eso cada una recibe todo.

SI NO LLEGA NINGÚN MENSAJE A NINGUNA TERMINAL

Comprueba desde la terminal T que los mensajes se escribieron:

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion --time -1
```

Los tres números del final tienen que sumar 6. Si suman 0, el productor no escribió. El motivo más común es haberlo lanzado desde una carpeta distinta o con el tópico mal escrito.

Otra brigada entra a la cocina

Qué vamos a hacer

Falta la segunda mitad de la afirmación. Hasta aquí todos los consumidores estaban en el mismo grupo.

¿Qué pasa con uno que declara un grupo distinto?

En SUNAT es un caso real. El equipo de **reportes** necesita leer todos los comprobantes para su analítica, y no puede quitarle trabajo al de validación.

NO CORTES NADA

Los consumidores `cons-A` y `cons-B` **siguen corriendo** en las terminales A y B. Déjalos. Usa la **terminal C**, que todavía está libre.

El comando, en la terminal C

```
docker exec -it kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion \
  --group reportes \
  --from-beginning \
  --consumer-property client.id=cons-Z \
  --property print.partition=true \
  --property print.key=true --property key.separator='|'
```

--group reportes — un nombre distinto. Eso es todo lo que cambia

--from-beginning — desde el principio del tópico, no solo lo que llegue de ahora en más. **Un grupo nuevo no tiene posición guardada**, así que hay que decirle desde dónde arrancar

client.id=cons-Z — el nombre de este consumidor. Le ponemos Z para que se note que es de otra brigada

LO QUE VAS A VER EN LA TERMINAL C

```
Partition:2|RUC-20100066604|comprobante_4
Partition:2|RUC-20100066606|comprobante_6
Partition:0|RUC-20100066602|comprobante_2
Partition:0|RUC-20100066605|comprobante_5
Partition:1|RUC-20100066601|comprobante_1
Partition:1|RUC-20100066603|comprobante_3
```

Los seis. Todos los comprobantes del laboratorio, en una sola terminal, incluidos los que el grupo `validacion` ya había leído y dado por procesados. `cons-Z` está solo en su brigada, así que se lleva las tres particiones y todo lo que hay dentro.

DOS COSAS SOBRE EL ORDEN

A ti el orden de las particiones te va a salir distinto. Lo que no cambia es que estén los seis y que salgan agrupados.

Salen agrupados por partición, no en orden de escritura, porque el consumidor recorre una partición y después otra. **Kafka garantiza el orden dentro de cada partición, nunca entre particiones.**

Y ahora lo que cierra el laboratorio, desde la terminal T

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group validacion
```

LO QUE VAS A VER · RECORTADO A LO ANCHO

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CLIENT-ID
validacion	novatech.validacion	0	2	2	0	cons-A
validacion	novatech.validacion	1	2	2	0	cons-A
validacion	novatech.validacion	2	2	2	0	cons-B

LO LOGRASTE SI...

El grupo `validacion` no se movió ni un offset. Que `reportes` haya leído los seis mensajes no le quitó nada.

Sigue con `CURRENT-OFFSET` igual a `LOG-END-OFFSET` en las tres particiones, y `LAG` en 0. **Cada brigada lleva su propia cuenta.**

SI EN LA TERMINAL C NO APARECE NADA

Casi siempre falta el `--from-beginning`. Sin él, un grupo nuevo arranca en «lo que llegue de ahora en adelante», y como ya nadie está escribiendo, no llega nada. Corta con `Ctrl+C` y relánzalo con el parámetro.

El techo · cuatro cocineros, tres sectores

ESTE ES EL PASO QUE HAY QUE ENTENDER

Si algo se te olvida de este laboratorio, que no sea esto. Es el concepto que más cuesta del paradigma, y el que decide cuánto hardware se compra.

Qué vamos a hacer

Volvemos a la pregunta que decide el presupuesto. Si dos validadores cortaron el atraso a la mitad, **¿ocho lo cortan a la octava parte?**

Kafka reparte **particiones**, no mensajes. Y hay tres. Vamos a poner cuatro consumidores en el mismo grupo y a mirar qué le toca a cada uno.

DE AHÍ SALE EL TECHO, Y ES ARITMÉTICA

Si Kafka reparte particiones, **nadie puede recibir media partición**. Un tópico de 3 particiones admite como mucho **3 miembros con trabajo**.

El cuarto entra al grupo, aparece en los listados, consume memoria, y no recibe nada.

Primero, libera la terminal C

En la **terminal C**, presiona `Ctrl+C` para cortar `cons-Z`. Ya cumplió su papel en el paso 5.

Ahora levanta dos consumidores más, en `validacion`

En la **terminal C**:

```
docker exec -it kafka-broker-1 kafka-console-consumer \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.validacion \  
  --group validacion \  
  --consumer-property client.id=cons-C \  
  --property print.partition=true \  
  --property print.key=true --property key.separator='|'
```

Y en la **terminal D**, el mismo comando con otro nombre:

```
docker exec -it kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion \
  --group validacion \
  --consumer-property client.id=cons-D \
  --property print.partition=true \
  --property print.key=true --property key.separator='|'
```

--group validacion — los cuatro en la misma brigada. Ahora hay cuatro miembros y tres sectores

Espera unos 30 segundos. El rebalanceo con cuatro miembros tarda más que con dos.

Pide la lista de miembros, desde la terminal T

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group validacion --members
```

--members — en vez de una línea por partición, **una línea por miembro**, con cuántas particiones tiene cada uno. **Es la vista que hace visible al que no tiene ninguna**

LO QUE VAS A VER · RECORTADO A LO ANCHO

GROUP	CONSUMER-ID	CLIENT-ID	#PARTITIONS
validacion	cons-C-83fad3cd-cb91-4d3b-8a29-e6f9a173b36d	cons-C	1
validacion	cons-A-bb74078d-12bc-4fb7-b94e-ad1b90995aed	cons-A	1
validacion	cons-B-525ed9ef-6195-41e2-b9d1-e7b0e0fd944f	cons-B	1
validacion	cons-D-440a17ba-b358-4d05-b82f-f7f80b864407	cons-D	0

Cuatro miembros. La columna **#PARTITIONS** dice **1, 1, 1 y 0**.

cons-D está en el grupo, está vivo, está conectado, y no va a recibir un solo mensaje. Su terminal se va a quedar en blanco para siempre. No está roto. Es que no hay una cuarta partición que darle.

A TI EL OCIOSO PUEDE SER OTRO

Puede tocarle a **cons-B**, a **cons-C**, al que sea, y las cuatro filas pueden salir en otro orden. **Lo que no cambia es que haya exactamente uno con 0.**

LA RESPUESTA A LA PREGUNTA DEL PRESUPUESTO

Con 3 particiones, el cuarto validador no acelera nada. Ni el quinto, ni el octavo. Pagas por ellos, aparecen en los tableros, y miran.

Y aquí se cierra el laboratorio 05 con este. El número de particiones se decide al crear el tópico y solo se puede **subir**, nunca bajar. O sea que **el día que creas un tópico estás fijando cuánto vas a poder paralelizar su consumo para siempre**. Esa es la razón real por la que la elección de particiones es la decisión más difícil de deshacer de Kafka.

LO LOGRASTE SI...

Salen **cuatro líneas** y en la columna `#PARTITIONS` hay **exactamente un cero**.

Confírmalo escribiendo tres mensajes más desde la terminal T y mirando la terminal del ocioso. **No va a llegar nada.**

SI TE SALEN MENOS DE CUATRO MIEMBROS

Alguno no llegó a unirse, o le diste poco tiempo al rebalanceo. **Espera 30 segundos más y repite el comando.**

Si sigue faltando uno, mira su terminal. Si ahí hay un error de conexión, córtalo con `Ctrl+C` y relánzalo.

SI EL GRUPO APARECE EN `PREPARINGREBALANCE` O SIMILAR

Es el estado normal mientras Kafka rehace el reparto. **No hagas nada.** Espera medio minuto y vuelve a preguntar.

El rebalanceo · se va un cocinero a mitad de turno

Qué vamos a hacer

En producción los procesos se caen, se reinician y se despliegan. La pregunta que importa es esta.

Cuando uno se va, ¿sus particiones quedan huérfanas?

CONCEPTO · REBALANCEO

El reparto que Kafka **rehace solo** cada vez que un miembro entra o sale del grupo. Nadie lo dispara a mano. Es el jefe de turno redistribuyendo los sectores cuando un cocinero se va.

Corta uno de los que sí tiene trabajo

Mira la salida del paso 6 y elige un consumidor con `#PARTITIONS` igual a **1**. Ve a su terminal y presiona `Ctrl+C`.

Eso es todo. El consumidor avisa que se va y termina. **Espera unos 20 segundos** y pregunta desde la terminal T:

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group validacion --members
```

LO QUE VAS A VER · RECORTADO A LO ANCHO

GROUP	CONSUMER-ID	CLIENT-ID	#PARTITIONS
validacion	cons-A-bb74078d-12bc-4fb7-b94e-ad1b90995aed	cons-A	1
validacion	cons-B-525ed9ef-6195-41e2-b9d1-e7b0e0fd944f	cons-B	1
validacion	cons-D-440a17ba-b358-4d05-b82f-f7f80b864407	cons-D	1

Quedan tres miembros y tres particiones. **El que estaba en cero ya no lo está.** Nadie ejecutó un comando de reasignación. Kafka rehízo el reparto solo, y el trabajo del que se fue lo tomó el que estaba mirando.

Compruébalo con tres comprobantes más, desde la terminal T

```
for i in 7 8 9; do
  echo "RUC-201006660${i}:comprobante_${i}" | \
  docker exec -i kafka-broker-1 kafka-console-producer \
    --bootstrap-server kafka-broker-1:29092 \
    --topic novatech.validacion \
    --property parse.key=true --property key.separator=:
done
```

Los tres aparecen repartidos entre las terminales que siguen vivas. **Ninguno se perdió**, y ninguno llegó dos veces.

Y EL QUE ANTES ESTABA OCIOSO PUEDE SEGUIR EN BLANCO

En la corrida con la que se midió esta guía, los tres comprobantes cayeron en dos particiones y el miembro recién ascendido no recibió ninguno. **Eso no lo contradice**. Ahora tiene una partición asignada, que es lo que se estaba demostrando. Con solo tres mensajes, que alguna partición se quede sin nada es lo normal.

LO LOGRASTE SI...

Ya no hay ningún miembro con #PARTITIONS en 0, y entre todas las terminales vivas sumas los tres comprobantes nuevos, sin repetidos.

Eso cierra el círculo del laboratorio. **El reparto lo hace Kafka, se rehace solo, y el techo lo pusiste tú al elegir el número de particiones.**

EL REBALANCEO TAMBIÉN ES UN COSTO

Nadie tiene que reasignar nada cuando un proceso se cae, y eso está muy bien. Pero **durante el rebalanceo el grupo para**. Si tus procesos se reinician seguido, estás pagando pausas que no ves en ningún tablero.

Lo que aprendiste

1 `--group` es la palabra más importante del comando.

Con el mismo nombre, los procesos colaboran. Con nombres distintos, duplican. Y por fuera se ven idénticos. El nombre del grupo es un dato de arquitectura escondido en una línea de configuración.

2 El paralelismo lo fijaste al crear el tópico.

Kafka reparte particiones, no mensajes. Más consumidores que particiones es plata gastada en procesos que miran. Y las particiones solo suben, nunca bajan.

3 El rebalanceo es automático, y también es un costo.

Nadie tiene que reasignar nada cuando un proceso se cae. Pero durante el rebalanceo el grupo para. Procesos que se reinician seguidos son pausas que estás pagando sin verlas.

4 `LAG` es el número que se vigila.

Es la única señal temprana de que el consumo va más lento que la producción. Cuando alguien pregunte «¿vamos atrasados?», es esto lo que se mira.

Para profundizar

Lo que sigue quedó fuera del recorrido de clase por tiempo. **Todos estos comandos corren tal cual están escritos** sobre el clúster que tienes arriba.

A · Consumir sin grupo

```
docker exec -it kafka-broker-1 kafka-console-consumer \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.validacion --from-beginning
```

sin **--group** — Kafka le inventa uno, del estilo `console-consumer-40f2d1ba`. **Ese consumidor recibe todo**

Lo que hay que mirar. Es el caso del paso 5 llevado al extremo. Cada terminal es su propia brigada. Y algo más incómodo, ese grupo inventado queda registrado en el clúster, así que un tablero de grupos de producción se llena de basura si la gente consume así.

B · Rebobinar un grupo

Es la operación que se hace cuando hay que reprocesar todo desde el principio.

AQUÍ EL ENVOLTORIO NO TE SIRVE

`kafka-cli/reset-group.sh` trae el tópico fijo por dentro, y es `novatech.fleet.events`. Si lo ejecutas después de este laboratorio, **rebobina un tópico que tu grupo nunca leyó** y deja intacto el `novatech.validacion` que sí trabajaste. Por eso aquí va el comando crudo, con su `--topic` explícito.

```
docker exec kafka-broker-1 kafka-consumer-groups \  
  --bootstrap-server kafka-broker-1:29092 \  
  --reset-offsets --group reportes \  
  --topic novatech.validacion \  
  --to-earliest --execute
```

--reset-offsets — mueve la posición guardada de un grupo. **No borra ni un mensaje**, solo cambia desde dónde va a leer la próxima vez

--group reportes — a qué grupo se le mueve la posición

--topic novatech.validacion — sobre qué tópico, con todas sus particiones

--to-earliest — al principio de todo. Hay más destinos, como `--to-latest` o `--to-datetime`

--execute — aplícalo de verdad. Sin esto solo te muestra lo que haría, que es la forma prudente de mirarlo primero

LO QUE SALE

GROUP	TOPIC	PARTITION	NEW-OFFSET
reportes	novatech.validacion	0	0
reportes	novatech.validacion	1	0
reportes	novatech.validacion	2	0

Y ahora míralo desde el otro lado:

```
docker exec kafka-broker-1 kafka-consumer-groups \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --group reportes
```

LO QUE SALE · RECORTADO A LO ANCHO

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
reportes	novatech.validacion	0	0	4	4
reportes	novatech.validacion	1	0	3	3
reportes	novatech.validacion	2	0	2	2

El **CURRENT-OFFSET** volvió a 0 y el **LAG** saltó al total. El grupo va a releer los nueve mensajes la próxima vez que alguien lo levante.

LA CONDICIÓN QUE HAY QUE CONOCER

El grupo tiene que estar sin miembros activos. Tras un **Ctrl+C** hay que esperar a que la sesión caduque, o el reset falla con «Assignments can only be reset if the group is inactive». En producción eso significa que **rebobinar un grupo exige parar los consumidores**, y eso hay que planificarlo.

C · Las claves y el particionado

La clave de un mensaje decide en qué partición cae. Y esa decisión es estable, así que dos mensajes con la misma clave caen **siempre** en el mismo sitio.

AQUÍ EL ENVOLTORIO TAMPOCO TE SIRVE

`kafka-cli/produce-event.sh` escribe en `novatech.fleet.events`, no en el tópic de este laboratorio. Sirve, pero tus consumidores no lo van a ver. Para comprobarlo sobre `novatech.validacion`, va el comando crudo.

Primero anota dónde está cada partición:

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion --time -1
```

Ahora escribe tres mensajes con la **misma clave**:

```
for v in alfa beta gamma; do
  echo "NVT-1001:evento_{$v}" | \
  docker exec -i kafka-broker-1 kafka-console-producer \
    --bootstrap-server kafka-broker-1:29092 \
    --topic novatech.validacion \
    --property parse.key=true --property key.separator=:
done
```

NVT-1001 — la clave, y es **la misma en los tres**. Ahí está el experimento

Y vuelve a mirar los offsets.

ANTES Y DESPUÉS · ESTO SALIÓ EN UNA CORRIDA REAL

antes	después
novatech.validacion:0:4	novatech.validacion:0:4
novatech.validacion:1:3	novatech.validacion:1:3
novatech.validacion:2:2	novatech.validacion:2:5 ← +3

Dos particiones no se movieron ni un offset. Los tres cayeron enteros en la 2, porque el *hash* de **NVT-1001** da esa y siempre va a dar esa. Y como cada partición la atiende un solo miembro del grupo, esos tres los lee **siempre el mismo**, y en orden.

Esta es la herramienta para garantizar que los movimientos de un mismo contribuyente se procesen ordenados. **Si el orden importa, la clave no es opcional.**

Y EL PRECIO QUE TIENE

Si unos pocos contribuyentes generan la mayor parte del tráfico y particionas por RUC, todo ese volumen cae en las mismas particiones. Se llama *hot partitioning*, y es **el precio de garantizar orden por clave**. No hay forma de tener las dos cosas.

D · El techo con seis particiones

El clúster trae creado `novatech.fleet.events`, con 6 particiones, y los envoltorios de `kafka-cli/` trabajan sobre él:

```
kafka-cli/consume-as-group.sh mi-grupo
kafka-cli/list-groups.sh
kafka-cli/describe-group.sh mi-grupo
```

Lo que hay que mirar. Con 6 particiones, para ver un miembro ocioso harían falta **siete** consumidores. Es el mismo experimento del paso 6, con el techo más alto. Sirve para comprobar que la regla no depende del número.

E · Verlo con los ojos

Abre **http://localhost:8090** y ve a *Consumers*. Ahí están los grupos, sus miembros y su `LAG`, en una pantalla.

Compara lo que muestra con lo que devolvió `kafka-consumer-groups --describe`. **Es la misma información**, y saber qué comando hay detrás de cada pantalla es lo que te deja resolver el día que la interfaz no esté.

F · El reporte del laboratorio

`plantillas/reporte-entregable.md`, con las respuestas de referencia en `soluciones/reporte-resuelto.md`. **Míralas después de intentarlo.**

Antes de cerrar

DEJA EL TERRENO LIMPIO

Corta con `Ctrl+C` todos los consumidores que sigan abiertos, en las cuatro terminales. Si los dejas, siguen corriendo y ocupando el clúster.

Después, desde la terminal T:

```
bin/stop-lab.sh
```

Y comprueba con `docker ps --filter name=kafka-broker` que no quede nada.

LO QUE TE LLEVAS

Cuando alguien proponga «levantemos más procesos para ir más rápido», ahora sabes cuáles son las dos preguntas que hay que hacer antes de aprobar el gasto. **¿Van en el mismo grupo? Y ¿cuántas particiones tiene el tópico?**